操作系统

操作系统服务与结构

周炽金

华东师范大学
软件工程学院

这一章回答哪些问题?

- **用户和程序能得到什么?** 系统服务、命令行、图形与触控界面。
- **程序怎样请求内核服务?** API、系统调用、参数与返回值。
- **程序怎样运行起来?** 编译、链接、装载与跨平台。
- **内核怎样组织?** 单体、分层、微内核、模块与混合结构。
- **系统怎样开始工作?** 构建、引导，以及运行后的调试与观测。

资料：[1] Operating System Concepts，第 2 章

复制文件需要哪些系统服务?

以命令 `cp source.txt target.txt` 为例:

- **启动程序**:找到并运行复制工具。
- **访问文件**:打开源文件,创建目标文件,读出并写入数据。
- **管理资源**:提供 CPU、内存和设备访问。
- **处理失败**:检查权限、文件是否存在、磁盘是否有空间。
- **报告结果**:向用户返回成功状态或错误信息。

资料:[1] Operating System Concepts,第 2 章

支持程序运行的服务

- **用户界面（User interface）**：接收命令、点击或触控输入。
- **程序执行（Program execution）**：装入程序，开始运行，正常或异常结束。
- **I/O 操作（I/O operations）**：读取输入、输出结果、访问设备。
- **文件系统操作（File-system manipulation）**：创建与删除、读写与定位、目录查询、属性与权限管理。

资料：[1] Operating System Concepts，第 2 章

通信与错误处理

- **通信（Communication）**：进程在本机或通过网络交换数据。
  - 共享内存：访问约定的共享区域，还需要同步。
  - 消息传递：通过管道、套接字等接口发送和接收数据。
- **错误检测（Error detection）**：识别并处理程序、设备和硬件报告的错误。
  - 例如非法访问、文件读取失败；系统可以返回错误、记录日志或终止进程。
- 处理方式取决于错误类型；操作系统并不能自动恢复所有错误。

资料：[1] Operating System Concepts，第 2 章

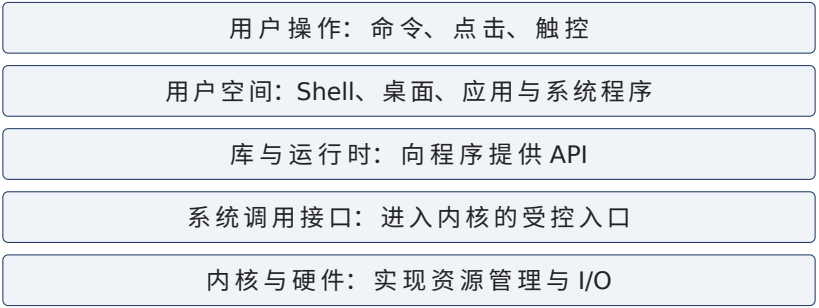
共享资源需要哪些服务?

- **资源分配（Resource allocation）**：分配 CPU 时间、内存、文件空间和设备。
- **记录与统计（Logging / Accounting）**：记录资源使用、系统事件与故障信息。
- **保护（Protection）**：规定并检查谁能访问什么，避免相互干扰。
- **安全（Security）**：身份认证、防范攻击、保护数据与系统。

例如：两个用户同时复制文件，系统既要分配资源，也要检查各自的访问权限。

资料：[1] Operating System Concepts，第 2 章

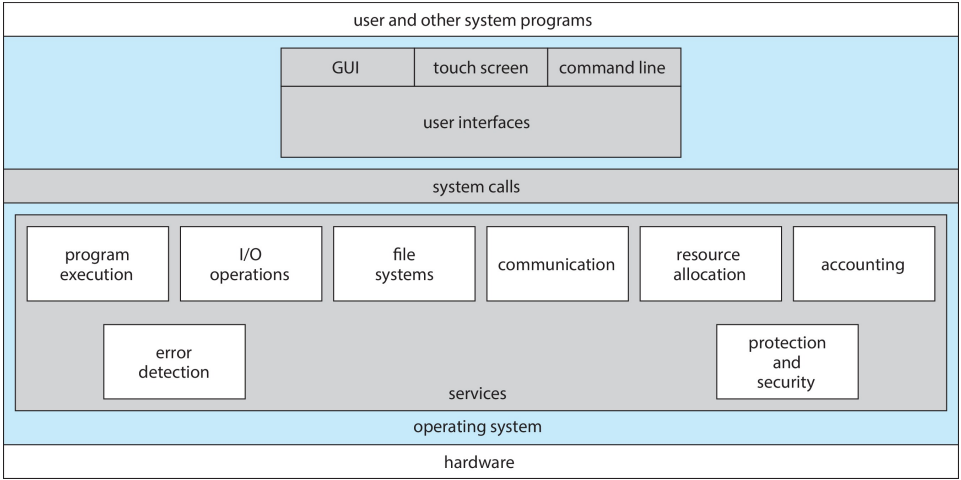
用户如何使用这些服务?



- 用户界面决定**怎样表达请求**；系统服务决定**系统提供什么能力**。
- 同一项文件操作，可以由不同界面发起。

资料：[1] Operating System Concepts，第 2 章

操作系统服务总览



资料：[1] Operating System Concepts，第 2 章

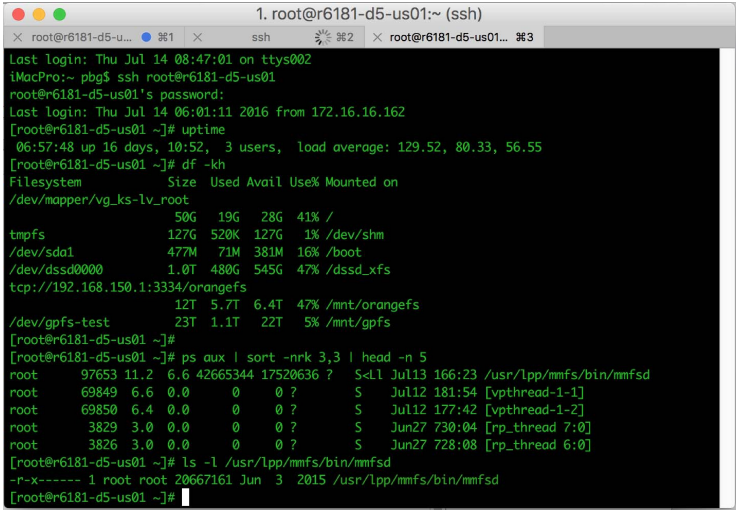
命令由谁解释?

```
pwd
cd /tmp
/bin/ls
```

- **Shell** 读取并解析命令，再执行内建操作或启动外部程序。
- **内建命令**：例如 Bash 的 `cd`，改变当前 Shell 的工作目录。
- **外部程序**：例如 `/bin/ls`；增加程序通常不必修改 Shell。
- Shell 是程序；在常见 Linux 桌面或服务器环境中运行于用户态。

资料：[1] Operating System Concepts，第 2 章　[7] GNU Bash：内建命令

命令行界面

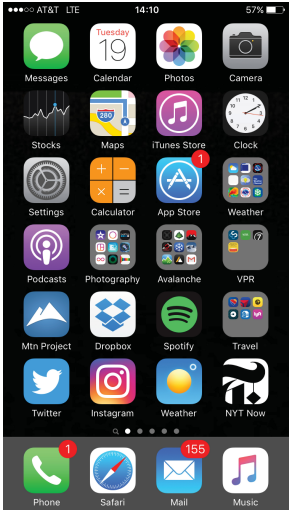

1. root@r6181-d5-us01:~ (ssh)
root@r6181-d5-u... ⌘1
ssh ⌘2
root@r6181-d5-us01... ⌘3
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
06:57:48 up 16 days, 10:52, 3 users, load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem Size Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
50G 19G 28G 41% /
tmpfs 127G 520K 127G 1% /dev/shm
/dev/sda1 477M 71M 381M 16% /boot
/dev/dssd0000 1.0T 480G 545G 47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs
12T 5.7T 6.4T 47% /mnt/orangefs
/dev/gpfs-test 23T 1.1T 22T 5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root 97653 11.2 6.6 42665344 17520636 ? S<Ll Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root 69849 6.6 0.0 0 0 ? S Jul12 181:54 [vpthread-1-1]
root 69850 6.4 0.0 0 0 ? S Jul12 177:42 [vpthread-1-2]
root 3829 3.0 0.0 0 0 ? S Jun27 730:04 [rp_thread 7:0]
root 3826 3.0 0.0 0 0 ? S Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x------ 1 root root 20667161 Jun 3 2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#

图形与触控界面

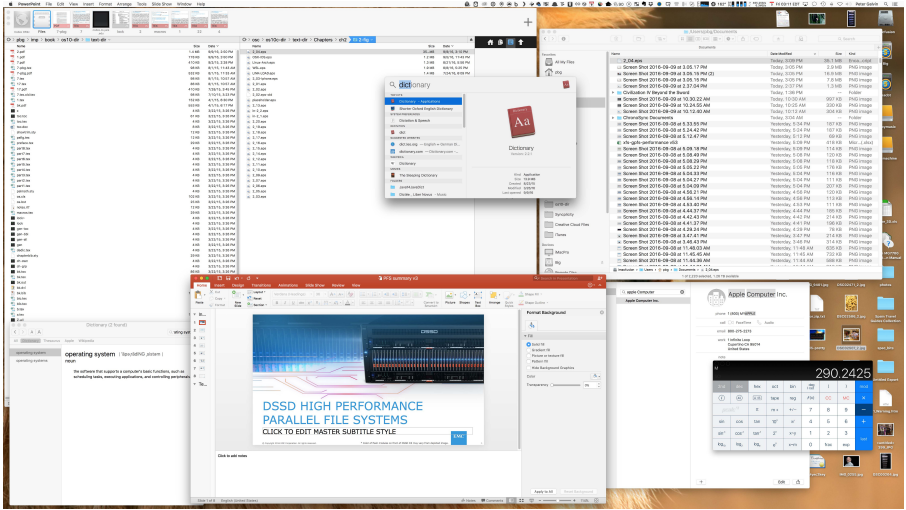
- **GUI（Graphical User Interface）**：窗口、图标、菜单，配合鼠标和键盘。
- **触控界面（Touch interface）**：点击、滑动等手势，以及虚拟键盘；也可结合语音。
- 图形界面的重要早期研究来自 **Xerox PARC**，后来在个人计算机上普及。
- Windows、macOS、Linux 桌面都可以同时提供 GUI 和命令行。
- 界面变化不改变基本路径：界面程序仍需通过系统提供的接口访问资源。

资料：[1] Operating System Concepts，第 2 章

手机的用户界面


●●●○○ AT&T LTE
14:10
57%
Messages
Tuesday
19
Calendar
Photos
Camera
Stocks
Maps
iTunes Store
Clock
Settings
Calculator
1
App Store
Weather
Podcasts
Photography
Avalanche
VPR
Mtn Project
Dropbox
Spotify
Travel
Twitter
Instagram
Weather
NYT Now
1
Phone
Safari
155
Mail
Music

Mac 的用户界面


dictionary
Dictionary
operating system
DSSD HIGH PERFORMANCE
PARALLEL FILE SYSTEMS
CLICK TO EDIT MASTER SUBTITLE STYLE
Apple Computer Inc.
290.2425

什么是系统调用?

System call：程序请求内核服务的受控接口。

*Programming interface to the **services provided by the OS**.*

- 读写文件、创建进程、建立通信等操作，需要系统提供服务。
- 应用通常调用**库或运行时的 API**，由它完成底层调用细节。
- 用户程序不能把内核函数当作普通函数随意跳转执行。
- 内核可以用 C 等语言实现；进入内核仍需要体系结构规定的入口。

资料：[1] Operating System Concepts，第 2 章　[2] syscall(2)

API、库函数与系统调用

概念	约定的对象	例子
API	源代码怎样调用功能	POSIX `read()`、Windows API、Java API
库或运行时实现	参数处理、缓冲、底层调用	C 标准库、JVM
系统调用	用户程序怎样受控进入内核	Linux 文件读写、进程操作

- 一个 API 调用可能对应**零次、一次或多次**系统调用。
- 例如字符串长度计算通常无需进入内核；文件输出可能先进入用户态缓冲区。

资料：[1] Operating System Concepts，第 2 章 [2] syscall(2) [5] stdio(3)

怎样读懂 read 接口?

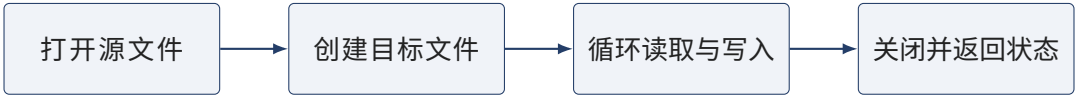
```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

- `fd`：从哪个已打开的文件描述符读取。
- `buf`：把数据放到哪里；`count`：最多读取多少字节。
- 对普通文件且 `count > 0`：返回正数表示实际读到的字节数，`0` 表示到达文件末尾。
- 返回 `-1` 表示失败，库设置 `errno`；**读得少不一定是出错**。

查询接口使用 `man 2 read`；Shell 的同名 `read` 是另一个接口。

资料：[3] read(2)

复制文件的调用过程



- 源文件不存在或无读取权限：报告错误。
- 目标文件的创建方式决定**覆盖已有文件还是拒绝覆盖**。
- 循环结束需要区分“读到末尾”和“读取失败”。
- 写入成功不等于断电后一定保留；持久性要求见第一课。

资料：[1] Operating System Concepts，第 2 章　[3] read(2)　[4] write(2)

调用返回后还要检查什么？

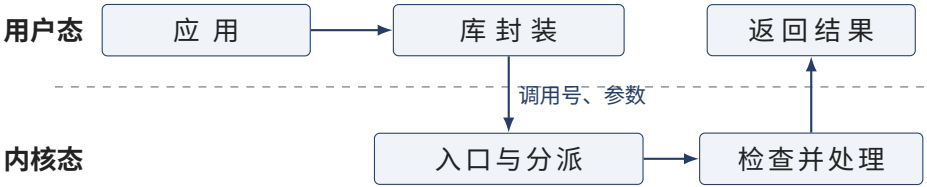
```
n = read(in, buf, sizeof buf);
/* n > 0: process data; n == 0: EOF; n < 0: error */

sent = 0;
while (sent < n) {
    k = write(out, buf + sent, n - sent);
    if (k <= 0) { /* retry or report error */ break; }
    sent += k;
}
```

- **短读、短写**：实际完成量可能小于请求量，依据返回值继续。
- **失败与中断**：先检查错误，再决定重试、退出或清理。
- 此处展示控制思路；完整处理见 `examples/copy_demo.c`。

资料：[3] read(2)　[4] write(2)

一次系统调用如何进入内核?

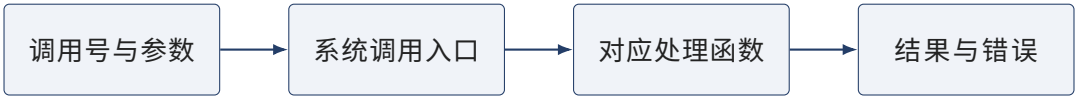


- 入口切换执行权限并保存必要状态，内核按约定处理请求。
- 调用结束后返回用户态；必要时也可能先等待或被调度。
- **进入内核不等于换了一个进程**。

资料：[1] Operating System Concepts，第 2 章　[2] syscall(2)

内核怎样找到要执行的操作?

- **调用号（System-call number）**标识所请求的操作。
- 入口读取调用号，检查范围，并分派到对应处理代码。
- **参数与返回约定**由操作系统和体系结构共同规定。
- 库封装隐藏寄存器等细节，并把结果转换成程序可用的形式。



资料：[1] Operating System Concepts，第 2 章　[2] syscall(2)

```c
// 计算斐波那契数列第 n 项
int fibonacci(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

int main() {
    // 计算第 10 个斐波那契数
    int result = fibonacci(10);

    // 把结果写入文件
    int fd = open("fib.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    char buf[50];
    int len = snprintf(buf, sizeof(buf), "Fibonacci(10) = %d\n", result);

    write(fd, buf, len);
    close(fd);

    printf("done!");

    return 0;
}
```

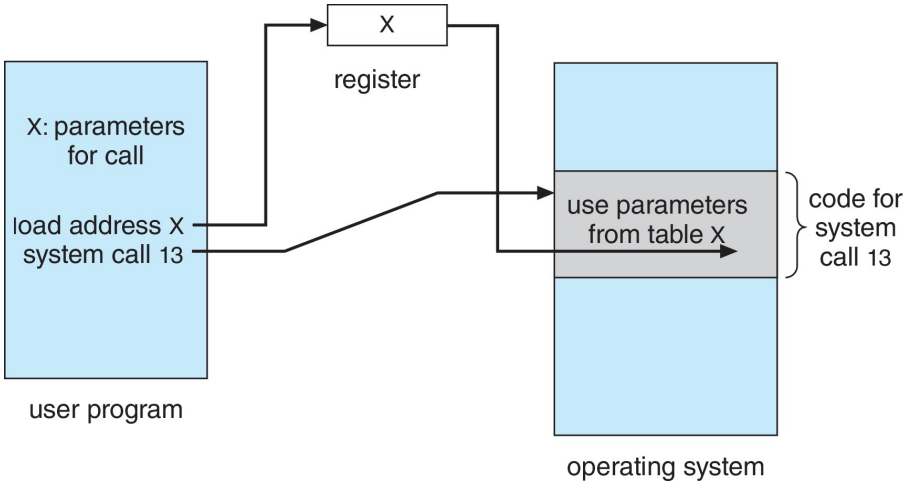
系统调用怎样传递参数?

一般方式	做法	需要注意
寄存器	把参数值放入约定寄存器	数量与布局受 ABI 约束
内存块	把参数结构的地址传给内核	内核检查地址并读取内容
栈	按约定把参数放在用户栈中	内核仍需检查长度与权限

- 这是三种一般办法，具体系统可以组合使用。
- Linux x86-64 的常见约定用寄存器传参；**指针指向的数据仍在内存中**。

资料：[1] Operating System Concepts，第 2 章 [2] syscall(2)

参数怎样交给内核



资料：[1] Operating System Concepts，第 2 章

为什么还要检查参数?

以 `read(fd, buf, count)` 为例:

- **对象是否有效?** `fd` 是否打开，是否允许读取?
- **内存是否可访问?** 返回数据时，内核是否可以写入该用户缓冲区?
- **长度是否合理?** 检查范围、边界及可能的整数溢出。
- **请求是否被授权?** 应用给出了路径或句柄，不等于自动获得权限。

系统调用把应用的请求交给内核，内核仍须验证请求。

资料: [2] syscall(2) [3] read(2)

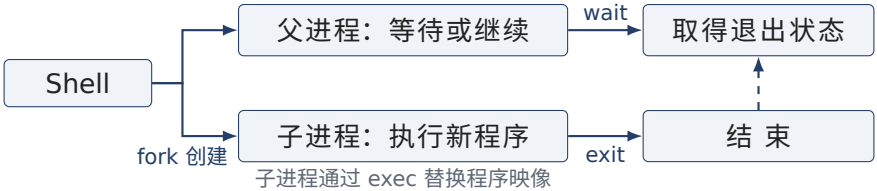
系统调用有哪些类别?

类别	主要解决的问题
进程控制	创建、执行、终止，等待与通知
文件管理	打开、读写、定位，目录与属性
设备管理	请求、配置、访问和释放设备
信息维护	查询或修改时间、标识和系统状态
通信	建立连接、传递消息、建立共享区域
保护	查询和设置权限，控制资源访问

分类帮助理解职责；一个接口可能同时涉及多个类别。

资料：[1] Operating System Concepts，第 2 章

进程控制



- Unix 常见接口：`fork()` 创建，`exec` 系列替换程序映像，`wait` 系列等待，`exit` 结束。
- 还包括进程属性、内存申请与释放、事件等待与通知。
- 调试、异常转储和同步，也需要相关的系统支持。

资料：[1] Operating System Concepts，第 2 章

文件与设备管理

- **文件生命周期**：创建、删除、打开、关闭。
- **数据与位置**：`read()`、`write()`、`lseek()`；查询或设置文件属性。
- **设备操作**：打开设备、读写或发出控制请求，查询和配置设备属性。
- **共同点**：都需要检查访问条件、处理错误并管理资源。
- **不同点**：设备可能不支持定位或某种读写；不能把所有设备当作普通磁盘文件。

资料：[1] Operating System Concepts，第 2 章 [3] read(2) [4] write(2)

信息、通信与保护

- **信息维护**：查询时间、进程标识、文件或设备属性；修改通常受权限限制。
- **消息传递**：建立管道或套接字，发送、接收并报告连接状态。
- **共享内存**：建立或映射共享区域，参与者另行协调访问。
- **保护**：查询与设置权限，允许或拒绝访问。

例如，文件读写、网络通信和共享内存都必须受到访问控制。

资料：[1] Operating System Concepts，第 2 章

Windows 与 Unix 的接口

任务	Unix / POSIX 常见接口	Windows 常见 API
创建或执行进程	`fork()`、`exec` 系列	`CreateProcess()`
等待结束	`wait()` / `waitpid()`	`WaitForSingleObject()`
文件读写	`read()` / `write()`	`ReadFile()` / `WriteFile()`
关闭资源	`close()`	`CloseHandle()`

- 这是**相似功能的应用接口对照**，不是语义一一相同的内核入口表。
- 例如 Windows `CreateProcess()` 同时创建进程和初始线程，不能直接等同于 `fork()`。

资料：[1] Operating System Concepts，第 2 章　[10] Microsoft：CreateProcessW

printf 是否就是 write?

```
setvbuf(stdout, NULL, _IOFBF, BUFSIZ);
printf("A");
printf("B\n");
fflush(stdout);
write(STDOUT_FILENO, "C\n", 2);
```

- `printf()` 进行格式化，结果可以先进入 **C 库缓冲区**。
- `fflush()` 推送缓冲内容；底层通常使用 `write()` 等接口。
- `write()` 成功表示内核接受了相应数据；**不保证磁盘持久化**。

资料：[4] write(2)　[5] stdio(3)

用 strace 观察系统调用

```
gcc -O0 -g -fno-builtin hello_io.c -o hello_io
strace -e trace=write ./hello_io
```

典型输出片段（参数与返回值）：

```
write(1, "AB\n", 3) = 3
write(1, "C\n", 2) = 2
```

- 左侧是调用及参数，右侧是返回值；它不直接显示 `printf()` 等普通库函数。
- `-f` 跟踪子进程，`-c` 汇总次数和耗时；跟踪本身也会影响运行。

资料：[5] stdio(3)　[6] strace(1)

一段实际的系统调用轨迹

```
fstat(3, {st_mode=S_IFREG|0755, st_size=2029592, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
pread64(3, "\4\0\0\0\20\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0", 32, 848) = 32
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\30x\346\264ur\f|Q\226\236i\253-'o"..., 68, 880) = 68
mmap(NULL, 2037344, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x784dad395000
mmap(0x784dad3b7000, 1540096, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x22000) = 0x784dad3b7000
mmap(0x784dad52f000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x19a000) = 0x784dad52f000
mmap(0x784dad57d000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e7000) = 0x784dad57d000
mmap(0x784dad583000, 13920, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x784dad583000
close(3)                                = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x784dad393000
arch_prctl(ARCH_SET_FS, 0x784dad393f40) = 0
mprotect(0x784dad57d000, 16384, PROT_READ) = 0
mprotect(0x784dad5a0000, 4096, PROT_READ) = 0
mprotect(0x784dad6ef000, 4096, PROT_READ) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x784dad391000
mprotect(0x784dad8c2000, 45056, PROT_READ) = 0
mprotect(0x403000, 4096, PROT_READ)     = 0
mprotect(0x784dad90c000, 4096, PROT_READ) = 0
munmap(0x784dad8d5000, 38642)           = 0
brk(NULL)                               = 0x1c0e9000
brk(0x1c10a000)                         = 0x1c10a000
openat(AT_FDCWD, "fib.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "Fibonacci(10) = 55\n", 19)    = 19
close(3)                                = 0
fstat(1, {st_mode=S_IFREG|0644, st_size=5031, ...}) = 0
write(1, "done!", 5done!)                     = 5
exit_group(0)                           = ?
+++ exited with 0 +++
```

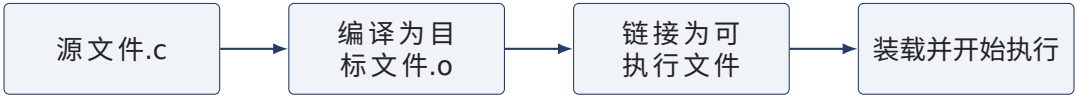
系统程序提供什么?

- **日常工具**：文件管理、文本编辑、状态查询与命令解释器。
- **开发工具**：编译器、链接器、调试器与程序装载支持。
- **通信与后台服务**：远程连接、日志、网络管理等。
- 系统程序通常运行在**用户空间**，通过接口使用内核服务。

例如：Shell、复制工具、编译器和日志服务都不是内核本身。

资料：[1] Operating System Concepts，第 2 章

从源代码到运行



- **编译（Compile）**：把源代码转换成目标代码。
- **链接（Link）**：合并目标文件，解析符号引用，并处理重定位。
- **装载（Load）**：建立运行所需的地址空间、映射与初始状态。
- 文件在磁盘上存在，不等于进程已开始执行。

资料：[1] Operating System Concepts，第 2 章　[8] elf(5)　[9] ld.so(8)

链接器在解决什么问题?

```c
/* main.c */
int add(int, int);
int main(void) { return add(2, 3) == 5 ? 0 : 1; }

/* add.c */
int add(int a, int b) { return a + b; }
```

- 编译 `main.c` 时可以生成对 `add` 的引用，而不必已知它的最终位置。
- 链接器找到 `add` 的定义，组合代码与数据，并修正有关引用。
- 可用 `nm main.o` 查看未定义符号，用 `readelf -r main.o` 查看重定位项。

资料：[1] Operating System Concepts，第 2 章 [8] elf(5)

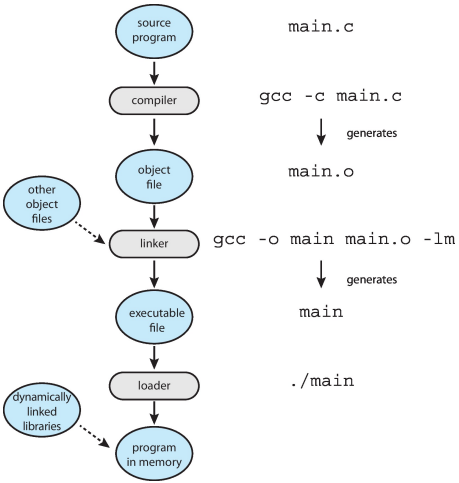
静态链接与动态链接

比较	静态链接	动态链接
库代码	所需代码并入可执行文件	运行时装入或映射共享库
部署依赖	通常减少外部库依赖	需要兼容的库与加载环境
更新影响	更新库后通常需重新链接	可更新共享库，但要保持兼容
空间使用	不同程序可能包含重复代码	可共享合适的只读代码页

- 两种方式都在使用，程序也可以混合使用。
- 共享库的可写数据通常仍各自隔离；动态链接不等于共享全部进程内存。

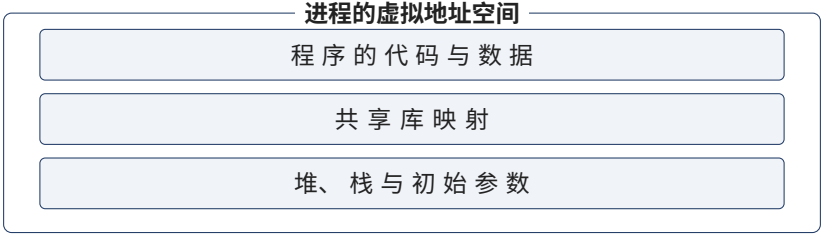
资料：[1] Operating System Concepts，第 2 章　[9] ld.so(8)

链接与装载的过程



资料：[1] Operating System Concepts，第 2 章

装载后，程序在哪里？



- 可执行文件格式说明需要映射的段、权限及程序入口。
- 内核与动态加载器协作准备运行环境；程序通常经启动代码到达 `main()`。
- 代码与数据可以按需调入内存，**不要求一次复制全部文件**。

资料：[8] elf(5)　[9] ld.so(8)

同一程序为什么不能直接跨系统运行?

- **指令集**：二进制指令要能被目标 CPU 执行。
- **文件格式**：目标系统要认识可执行文件的格式和装载约定。
- **系统与库接口**：调用的功能、版本和运行时需要匹配。
- **ABI（Application Binary Interface）**：二进制层面的调用约定、数据布局等。

API 面向源代码的使用约定；ABI 关系到编译后的代码能否相互配合。

资料：[1] Operating System Concepts，第 2 章　[2] syscall(2)　[8] elf(5)

跨平台有哪些办法?

办法	程序怎样运行	仍需满足什么条件
解释器	Python 等由目标平台解释器执行	依赖库、平台 API、路径等兼容
虚拟机 / 运行时	Java 字节码由 JVM 等执行	运行时版本与本地依赖兼容
分别编译	C/C++ 源码针对各平台构建	源码可移植，适配平台差异

跨平台需要可用的运行环境；语言相同不代表程序自动兼容。

资料：[1] Operating System Concepts，第 2 章

操作系统设计要权衡什么？

- **用户目标**：方便使用、响应及时、运行可靠。
- **系统目标**：有效利用资源，便于维护、移植和扩展。
- **实现约束**：处理器、内存、设备、应用兼容性与开发成本。
- **实现语言**：高级语言便于组织复杂系统，底层入口仍可能需要汇编。

例如：更强的隔离可能增加通信开销；更丰富的功能也会增加维护成本。

资料：[1] Operating System Concepts，第 2 章

机制与策略有什么区别?

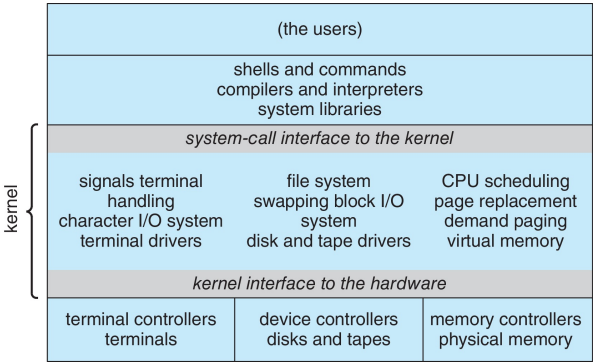
- **机制（Mechanism）**：提供“能够怎样做”的手段。
- **策略（Policy）**：决定“何时做、选谁、做多少”。

场景	机制	策略
CPU 调度	时钟中断、保存和恢复执行状态	下一次运行哪个任务
内存分配	页表与映射、分配和回收操作	给谁多少、何时回收

把机制和策略适当分离，便于改变决策而少改动底层实现。

资料：[1] Operating System Concepts，第 2 章

单体内核



- **单体（Monolithic）**：许多核心服务位于同一内核地址空间，直接调用协作。
- 优点是内部交互方便；错误也可能影响整个内核。
- 传统 UNIX 是典型例子；单体内核仍可以有清晰的模块和接口。

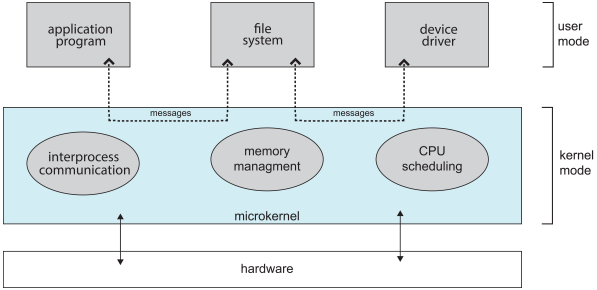
资料：[1] Operating System Concepts，第 2 章

分层结构

- **分层（Layered）**：每层建立在较低层的服务之上，并向上提供接口。
- 例如：硬件 → 硬件相关机制 → 资源管理与抽象 → 用户接口与服务。
- 便于理解依赖、局部验证与维护。
- 难点是合理划分边界；逐层调用可能增加开销或限制优化。
- 分层描述依赖关系，**不等于每层都有独立的进程或权限级别**。

资料：[1] Operating System Concepts，第 2 章

微内核



- **微内核（Microkernel）**：把更多服务放到用户空间，通过 IPC 协作。
- 更小的内核有利于隔离和验证；服务可更独立地替换或重启。
- 跨地址空间通信与切换有成本，具体性能取决于设计和实现。
- Mach 是经典例子；较少的内核代码并不自动保证系统无漏洞。

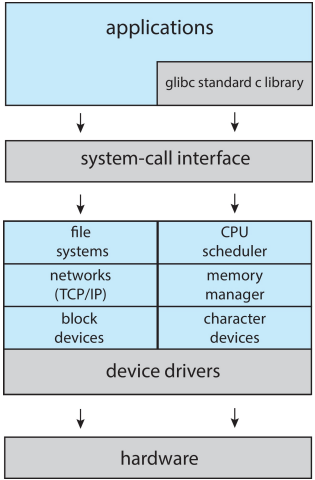
资料：[1] Operating System Concepts，第 2 章

模块化与 Linux

- **模块化（Modular）**：以明确接口组织功能，部分组件可动态装入和卸载。
- Linux 内核结合了**单体内核与模块化**：许多服务仍在内核空间运行。
- 驱动或文件系统可以是内建功能，也可以配置为可加载模块。
- 模块加载后拥有内核执行权限；模块化不等于微内核的进程隔离。

资料：[1] Operating System Concepts，第 2 章　[15] Linux：External Modules

Linux 内核中的各个部分



资料：[1] Operating System Concepts，第 2 章

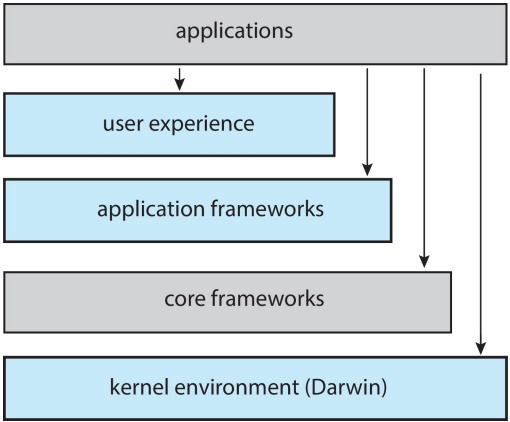
混合结构怎样理解?

结构思路	主要关注什么
单体	多数核心服务是否同处内核空间
分层	功能依赖是否按层次组织
微内核	哪些服务移到用户空间并经 IPC 协作
模块化	功能能否通过明确接口组合、替换或加载

- 这些维度可以组合，实际系统常采用混合设计。
- Windows NT 系列、Apple XNU 常用于讨论混合结构；应看实际边界和调用路径。

资料：[1] Operating System Concepts，第 2 章 [12] Apple：XNU

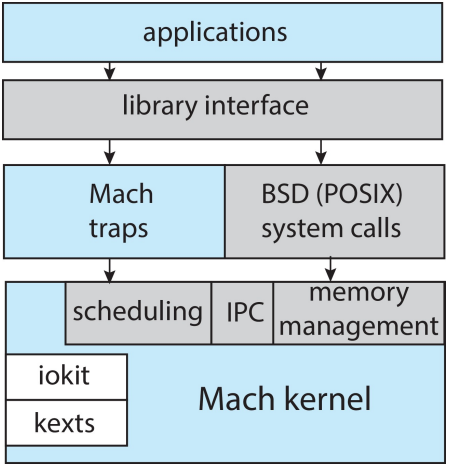
macOS、iOS 与 Darwin



- **Darwin** 包括 XNU 与基础系统组件；XNU 结合 Mach、BSD 和 I/O Kit。
- 共享底层技术不等于应用二进制兼容；平台框架、ABI 和设备能力仍不同。
- 移动设备还要考虑触控、功耗与应用生命周期。

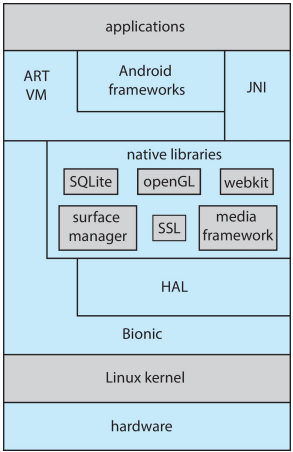
资料：[1] Operating System Concepts，第 2 章 [11] Apple：Kernel Architecture [12] Apple：XNU

Darwin 的内核结构



资料：[1] Operating System Concepts，第 2 章 [11] Apple：Kernel Architecture [12] Apple：XNU

Android 的系统结构



- AOSP 提供 Android 开源平台；具体设备还可包含厂商组件。
- **ART** 是当前 Android 运行时；**Dalvik** 属于早期实现。
- Java/Kotlin 应用使用 Android API；原生 C/C++ 代码可经 NDK 参与。
- Android 使用 Linux 内核，并不等于桌面 Linux 的完整软件环境。

资料：[13] AOSP：Architecture [14] AOSP：ART / Dalvik

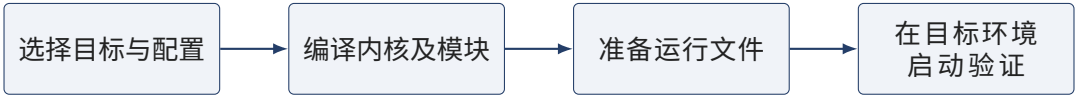
一个最小的内核模块

```
static int __init hello_init(void) {
    pr_info("lesson2: loaded\n");
    return 0;
}
static void __exit hello_exit(void) {
    pr_info("lesson2: unloaded\n");
}
module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

- 模块入口完成初始化；退出函数释放初始化时取得的资源。
- 通过内核构建系统生成 `.ko`，需要匹配的配置、头文件和内核接口。

资料：[15] Linux：External Modules

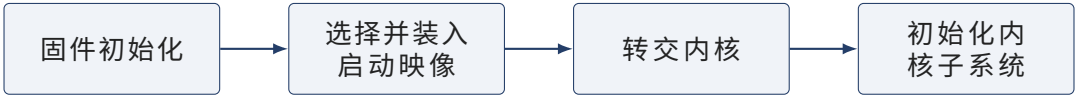
怎样生成操作系统?



- 配置处理器架构、驱动与功能：**内建、模块或不启用**。
- 构建得到内核映像、模块等产物；根文件系统还需用户空间程序与库。
- 同一套源码可产生面向不同硬件和用途的系统。
- 构建成功后，还要验证启动、设备支持和应用运行。

资料：[1] Operating System Concepts，第 2 章 [15] Linux：External Modules [16] Linux：Kconfig

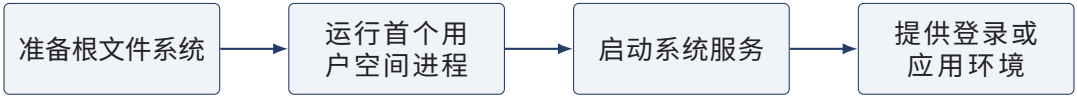
从开机到内核



- **固件（Firmware）**先执行，完成必要硬件初始化并选择启动路径。
- **引导程序（Boot loader）**准备内核、参数及可能的初始文件系统。
- 内核接管后初始化内存管理、中断、调度与必要驱动。
- 某些 UEFI 路径可直接装入内核映像；启动链不总有相同数量的阶段。

资料：[1] Operating System Concepts，第 2 章 [17] systemd-boot [18] Linux：init

内核启动以后呢？



- Linux 最终需要运行用户空间的 `init`；常见实现之一是 systemd。
- 初始内存文件系统可以先运行 `/init`，再切换到实际根文件系统。
- 后台服务负责日志、网络等；桌面或 Shell 为用户提供入口。
- “内核已运行”和“用户已能使用整套系统”是不同阶段。

资料：[1] Operating System Concepts，第 2 章 [18] Linux：init

调试与性能分析

问题	需要的证据	常用工具或产物
结果错误、程序崩溃	出错位置、变量与调用栈	日志、GDB、core dump
内核故障	内核日志与崩溃状态	内核日志、crash dump
程序运行缓慢	CPU、等待与 I/O 的分布	top、vmstat、perf、strace

- **调试（Debugging）**先回答“哪里错了”；性能分析回答“时间和资源用在哪里”。
- 从可复现的现象出发，保留命令、输入和运行环境。

资料：[1] Operating System Concepts，第 2 章 [19] perf-stat(1) [20] GNU GDB：Backtrace

性能数字应该怎样看?

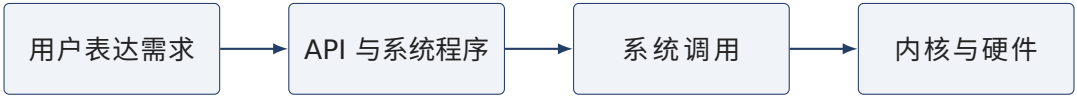
- **计数（Counters）**：运行期间发生多少次事件，例如上下文切换或缺页。
- **采样（Sampling）**：定期观察正在执行的位置，估计热点分布。
- **跟踪（Tracing）**：记录事件及顺序，例如系统调用轨迹。

```
strace -c ./hello_io
perf stat ./hello_io
```

- 先比较相同工作量与环境；工具有开销，权限也可能限制可见事件。
- CPU 占用低不代表没有瓶颈，程序可能在等 I/O 或锁。

资料：[6] strace(1) [19] perf-stat(1)

把这些概念连起来



- 程序运行前，还要经过**编译、链接与装载**。
- 内核内部用不同结构组织服务，权衡效率、隔离和可维护性。
- 系统先经过**构建与启动**，运行后用观测工具检查行为。
- 后续课程逐一展开进程、内存、文件与设备等服务的实现。

课堂练习

- 调用 `printf()` 是否一定立刻发生一次 `write()`？依据是什么？
- 一个函数参数是用户指针，内核为什么不能直接信任它？
- 两台机器使用相同 CPU，为什么仍可能不能运行同一个可执行文件？
- Linux 可以加载模块，是否就属于微内核？请说明判断依据。
- 从按下电源到出现 Shell，中间至少经过哪些阶段？

参考资料

[1] Silberschatz, Galvin, Gagne (2018) —Operating System Concepts, 10th ed., Chapter 2: Operating-System Services

[2] Linux man-pages —syscall(2): indirect system call

[3] Linux man-pages —read(2): read from a file descriptor

[4] Linux man-pages —write(2): write to a file descriptor

[5] Linux man-pages —stdio(3): standard input/output library functions

[6] strace —strace(1): trace system calls and signals

[7] GNU Bash Reference Manual —Bourne Shell Builtins

[8] Linux man-pages —elf(5): format of Executable and Linking Format files

[9] Linux man-pages —ld.so(8): dynamic linker/loader

[10] Microsoft Learn —CreateProcessW function

[11] Apple —Kernel Programming Guide: Kernel Architecture Overview

[12] Apple Open Source —XNU kernel README

[13] Android Open Source Project —Architecture overview

[14] Android Open Source Project —Android runtime and Dalvik

[15] The Linux Kernel documentation —Building External Modules

[16] The Linux Kernel documentation —Configuration targets and editors

[17] systemd —systemd-boot UEFI Boot Manager

[18] The Linux Kernel documentation —Explaining the “No working init found.” boot hang message

[19] Linux perf —perf-stat(1): run a command and gather performance counter statistics

[20] GNU GDB —Debugging with GDB: Backtrace